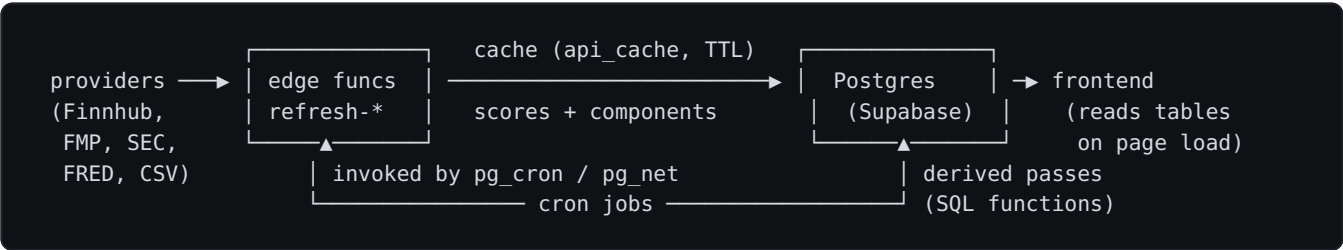


NEWNFL — How the data updates

This explains where every number on the site comes from, **how it gets refreshed, how often, and how fast**. The whole system is "pull on a schedule, cache aggressively, serve from the database" — the frontend never calls a data provider directly.



The three clocks

Everything is driven by **three pg_cron jobs** plus derived SQL passes.

Job	Schedule	What it does	Speed
<code>newnfl-refresh-stocks</code>	every minute	Refreshes the 10 stalest tickers' fundamentals + scores	~2–5 s per stock
<code>newnfl-cross-sectional</code>	every minute	Recomputes all relative-value metrics across the universe	< 1 s for all 503
<code>newnfl-refresh-regime</code>	daily, 09:35 UTC	Pulls macro series from FRED, scores the regime, caches the 10Y Treasury	~3–5 s

`recompute_universe_stats()` (quadrant median boundaries) runs **after every stock refresh**, so the BUY/WATCH/AVOID/SELL lines re-center continuously.

1. Per-stock fundamentals & scores

What: quality score, value score, and every component behind them (Piotroski, Altman, ROIC, DCF, Graham, dividend, etc.) — per ticker, in `quality_scores` / `value_scores`.

- **Sources:** Finnhub (financials-reported + real-time quote), FMP (key-metrics, ratios, DCF), SEC EDGAR (CIK fallback). Responses are cached in `api_cache` with a TTL (financial statements 24 h, price short-lived), so most refreshes are cache hits and only hit the network when data is actually stale.
- **Trigger:** `newnfl-refresh-stocks` calls `enqueue_stale_refreshes(10)` every minute. It picks the 10 tickers with the oldest `last_refreshed`, fires an async `refresh-stock/<ticker>` call for each via `pg_net`, and stamps `last_refreshed`.
- **Cadence:** 10 stocks/min, oldest-first → the full **~503-name universe cycles in ≈ 50 minutes**, then repeats. So any single stock is at most ~50 min stale.

- **Rate limits:** paced under Finnhub's 60 req/min (10 stocks × ~4 calls ≈ 40/min). Per-provider daily budgets are enforced by `consume_api_quota`.
- **Speed:** a single refresh is ~2–5 s end-to-end (mostly cached reads).

2. Relative-value metrics (the cross-sectional pass)

What: the metrics that only make sense by comparing a stock to the rest of the universe — **sector-median P/E**, **FCF-yield percentile**, **EV/EBITDA peer percentile**, and **earnings-yield-vs-10Y-Treasury spread** — plus the value composite that depends on them.

- **Source:** derived in pure SQL from columns already on `value_scores` (no external calls). The Treasury comes from the macro cache row (see §3).
- **Trigger / cadence:** `newnfl-cross-sectional` runs `recompute_cross_sectional()` **every minute**, recomputing all 503 stocks at once.
- **Speed:** sub-second for the whole universe.
- **Why a separate pass:** a single stock can't know its sector's median or its percentile rank until every stock is scored. This pass also **self-heals** — a per-stock refresh momentarily blanks these fields, and the next minute's run refills them.

3. Macro regime & the 10Y Treasury

What: the `regime` row (cycle phase, recession probability, composite, and the six indicator values/scores) and the cached 10Y Treasury used for the earnings-yield-vs-bond spread.

- **Source:** **FRED** (St. Louis Fed) — 7 series: real GDP, unemployment, CPI, fed funds, 10Y & 2Y Treasury, nonfarm payrolls. Free and effectively unlimited.
- **Trigger / cadence:** `newnfl-refresh-regime` runs **once daily at 09:35 UTC**. Macro data moves slowly (monthly/quarterly prints), so daily is plenty. Each series is cached 24 h.
- **Speed:** ~3–5 s. The result is one `regime` row + one `av:treasury_10y` cache row (tagged with the `_macro` sentinel ticker so it stays single-row).

4. Universe membership

What: which tickers exist and which indexes they belong to (`watchlist`, `index_membership`) — powers the S&P 500 / Dow / Nasdaq-100 toggles.

- **Source:** a public S&P 500 constituents CSV (live, with GICS sector + SEC CIK); Dow & Nasdaq-100 are version-controlled constants.
- **Trigger:** `refresh-universe`, run **on demand** (membership changes rarely).
- **Speed:** ~2–3 s.

5. The frontend

- Reads the tables **directly from Supabase on page load** (REST). There's no server render and no provider calls from the browser.

- The deep-dive modal **lazy-loads** `component_detail` for one stock when opened.
- **Data changes need no redeploy** — a new value in the database shows on the next page load. The site is only rebuilt when the React app itself changes.

End-to-end freshness, at a glance

Data	Updated by	How often	Full-universe latency
Fundamentals & raw scores	<code>refresh-stock</code> (cron)	continuous, 10/min	≤ ~50 min
Sector P/E, FCF/EV percentiles, EY-vs-bond	<code>recompute_cross_sectional</code>	every minute	< 1 s
Quadrant median boundaries	<code>recompute_universe_stats</code>	after each refresh	instant
Macro regime + Treasury	<code>refresh-regime</code> (cron)	daily 09:35 UTC	~5 s
Index membership	<code>refresh-universe</code>	on demand	~3 s
What the user sees	Supabase REST	every page load	real-time

Net effect: relative-value numbers and quadrant boundaries are never more than ~1 minute stale; a given stock's fundamentals are never more than ~50 minutes stale; macro refreshes daily.

Forcing an update manually

All refreshers are HTTP endpoints (JWT-protected) and the derived passes are RPCs:

```
-- One stock now (async, via pg_net):
select net.http_post(
  url := 'https://<project>.supabase.co/functions/v1/refresh-stock/AAPL',
  headers := jsonb_build_object('Authorization', 'Bearer <anon-key>', 'Content-Type', 'application/
json'),
  body := '{}':::jsonb);

-- A bigger batch of the stalest names:
select public.enqueue_stale_refreshes(40);

-- Recompute all relative-value metrics immediately:
select public.recompute_cross_sectional();

-- Refresh macro/regime now:
select net.http_post(
  url := 'https://<project>.supabase.co/functions/v1/refresh-regime',
  headers := jsonb_build_object('Authorization', 'Bearer <anon-key>', 'Content-Type', 'application/
json'),
  body := '{}':::jsonb);
```

Keep manual batches modest ($\leq \sim 40$ at a time) so per-stock fetches stay under Finnhub's 60 req/min ceiling.